UNIVERSIDADE FEDERAL DO CEARÁ

Test Smells em projetos OpenSource

Disciplina: Qualidade de Software

Equipe:

- Carla Cristina Sousa Araújo – 571600
- Kaylanne de Jesus Sousa Sátiro - 567741

Projeto: https://github.com/virgiliojr94/book-to-skill

Fork: https://github.com/kaylannesatiro/book-to-skill

Aluna: Kaylanne de Jesus Sousa Sátiro

A01 – Conditional Test Logic

Eu estou atualmente trabalhando na refatoração do code smell Conditional Test Logic no teste tests/test_batch_resilience_unreadable.py::test_batch_survives_post_extraction_stat_failure, linha 126, correspondente ao commit 4416f03. A ocorrência estava em uma função simulada que utilizava um if para decidir quando lançar PermissionError.
Minhas principais dificuldades na remoção do code smell são: Minha principal dificuldade foi preservar a ordem das respostas esperadas de os.path.getsize. A primeira consulta deveria representar o arquivo que ficou indisponível e lançar a exceção, enquanto a chamada seguinte deveria retornar normalmente o tamanho do arquivo válido.
Usei o CATSPY para: Consultei a página "Conditional Test Logic" do CATSPY: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic. Utilizei principalmente as seções de definição, detecção, possível efeito e refatoração. A página explica que condições e laços podem alterar o caminho executado pelo teste e dificultar a compreensão do resultado esperado. Comparei essa descrição com o if presente em flaky_getsize e utilizei a orientação

de retirar o fluxo condicional do teste, adaptando-a para um Mock com respostas sequenciais.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

O método de refatoração utilizado foi substituir a função condicional por um Mock com side_effect em sequência. O mock passou a lançar PermissionError na primeira chamada e retornar o tamanho do arquivo válido na segunda, preservando o cenário original sem manter o if.

A02 – Conditional Test Logic

Eu estou atualmente trabalhando na refatoração do code smell Conditional Test Logic no teste tests/test_batch_resilience_unreadable.py::test_batch_survives_post_extraction_stat_failure, linha 134, correspondente ao commit 1ddbe53. A ocorrência era um laço que aplicava as mesmas configurações aos módulos config e utils.

Minhas principais dificuldades na remoção do code smell são:

Minha principal dificuldade foi garantir que todas as seis alterações realizadas pelo laço continuassem presentes: OUTPUT_DIR, OUTPUT_TEXT e OUTPUT_META precisavam ser configurados separadamente nos dois módulos.

Usei o CATSPY para:

Consultei a página “Conditional Test Logic” do CATSPY: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic. A definição e a regra de detecção indicam a presença de instruções condicionais ou laços dentro do teste. Relacionei essa descrição ao laço que configurava config e utils. A seção de refatoração recomenda separar ou tornar explícitos os diferentes casos, o que foi adaptado para chamadas individuais de monkeypatch.setattr.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

O método de refatoração foi desdobrar o laço em seis chamadas diretas de monkeypatch.setattr. As três configurações foram aplicadas explicitamente em config e depois em utils. O diretório e os arquivos temporários permaneceram os mesmos.

A03 – Conditional Test Logic

Eu estou atualmente trabalhando na refatoração do code smell Conditional Test Logic no teste tests/test_batch_resilience_unreadable.py::test_batch_survives_unreadable_source, linha 64, correspondente ao commit df14561. A ocorrência era um laço usado para configurar os diretórios e arquivos de saída nos módulos config e utils.

Minhas principais dificuldades na remoção do code smell são:

Minha principal dificuldade foi ambiental. No Windows, a alteração de permissão empregada pelo teste não torna necessariamente o arquivo ilegível. Esse comportamento já existia antes da refatoração e o teste possui o tratamento skip_if_readable_anyway. Também preservei todas as configurações originalmente feitas pelo laço.

Usei o CATSPY para:

Consultei a página “Conditional Test Logic” do CATSPY: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic. Usei a definição para identificar o laço empregado na preparação dos dois módulos e a seção de possível efeito para avaliar se a alteração poderia produzir configurações diferentes entre config e utils. A recomendação de explicitar os diferentes casos foi aplicada por meio do desdobramento do laço, preservando os três atributos configurados em cada módulo.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

O método utilizado foi desdobrar o laço em chamadas explícitas de monkeypatch.setattr

para os dois módulos. A refatoração não modificou o mecanismo de arquivo ilegível nem o tratamento específico do Windows.

A04 – Conditional Test Logic

Eu estou atualmente trabalhando na refatoração do code smell Conditional Test Logic no teste tests/test_book_to_skill.py::TestBatchResilience::test_batch_continues_past_bad_files, linha 309, correspondente ao commit 14f159b. O teste percorria uma lista contendo um arquivo válido e um arquivo inválido e decidia o resultado por meio de um laço com tratamento de exceção.

Minhas principais dificuldades na remoção do code smell são:

Minha principal dificuldade foi separar os dois caminhos sem alterar o objetivo do teste. O arquivo válido deveria continuar sendo extraído, enquanto o arquivo inválido deveria continuar gerando um registro de erro.

Usei o CATSPY para:

Consultei a página "Conditional Test Logic" do CATSPY: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic. A página informa que o fluxo condicional pode fazer um teste executar caminhos diferentes e tornar seu resultado menos previsível. Relacionei essa descrição ao laço que tratava, no mesmo fluxo, um arquivo válido e outro inválido. A orientação de separar os casos foi adaptada para executar explicitamente a extração bem-sucedida e, em seguida, o cenário que produz ExtractionError.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

O método utilizado foi transformar os dois casos em operações lineares. Primeiro executei explicitamente a extração do arquivo válido. Depois executei separadamente o arquivo

inválido, mantendo a captura de ExtractionError. As listas extracted e errors continuaram com um elemento cada.

A05 – Conditional Test Logic

Eu estou atualmente trabalhando na refatoração do code smell Conditional Test Logic no teste tests/test_book_to_skill.py::TestDetectStructure::test_detects_chapter_word_with_roman_ numeral, linha 605, correspondente ao commit 756acfa. A entrada do teste era criada por uma expressão geradora que percorria os numerais romanos de I a V.
Minhas principais dificuldades na remoção do code smell são: Minha principal dificuldade não foi na implementação, mas na validação. Inicialmente usei o identificador pytest TestChapterDetection, porém a classe correta era TestDetectStructure. Corrigi o identificador e executei novamente o teste.
Usei o CATSPY para: Consultei a página "Conditional Test Logic" do CATSPY: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic. A regra de detecção inclui laços utilizados dentro do teste. Comparei essa regra com a expressão geradora responsável por criar os capítulos de I a V. A seção de refatoração foi usada como referência para retirar a construção iterativa e tornar o cenário de entrada explícito, mantendo os mesmos cinco capítulos e o mesmo resultado esperado.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: O método de refatoração foi substituir a expressão geradora por um texto explícito contendo os cinco capítulos. Mantive os mesmos títulos, numerais romanos e corpos de texto. O resultado esperado continuou sendo a detecção de cinco capítulos.

A06 – Conditional Test Logic

Eu estou atualmente trabalhando na refatoração do code smell Conditional Test Logic no teste tests/test_book_to_skill.py::TestDetectStructure::test_detects_persian_word_chapters_1_to_34, linha 1026, correspondente ao commit b80addb. O teste utilizava uma compreensão com enumerate para associar os 34 ordinais persas aos números de 1 a 34.
Minhas principais dificuldades na remoção do code smell são: Minha principal dificuldade foi preservar exatamente a ordem dos pares. Uma inversão entre o ordinal e o número alteraria o texto sintético e poderia invalidar o cenário.
Usei o CATSPY para: Consultei a página "Conditional Test Logic" do CATSPY: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic. A definição permitiu relacionar o smell à compreensão que combinava os ordinais persas com os números de 1 a 34. Também considerei o possível efeito descrito pelo catálogo, relacionado à compreensão e à manutenção. A refatoração do projeto não repetiu literalmente o exemplo do CATSPY: a iteração foi reexpressa com zip e starmap, preservando todos os pares originais.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: O método adotado foi combinar zip, range(1, 35) e itertools.starmap. A nova construção manteve os mesmos 34 pares e produziu o mesmo texto. Considero essa alteração principalmente uma reestruturação sintática, pois não existe evidência de uma melhora ampla de legibilidade.

A07 – Conditional Test Logic

Eu estou atualmente trabalhando na refatoração do code smell Conditional Test Logic no teste

tests/test_book_to_skill.py::TestDocxExtraction::test_extract_docx_python_docx_absent_skips_validation_without_raising, linha 1627, correspondente ao commit c37f5a8. O teste possuía uma função fake_import com um if que verificava se o módulo importado era docx.

Minhas principais dificuldades na remoção do code smell são:

Minha principal dificuldade foi simular a ausência do módulo docx sem alterar o comportamento que o teste pretendia verificar. A função original interceptava as importações e utilizava uma condição para lançar ImportError somente quando o módulo solicitado era docx. Ao retirar essa função, precisei garantir que a importação continuasse falhando no ponto esperado e que extract_docx_with_python_docx continuasse retornando None, sem tentar processar o arquivo inválido.

Usei o CATSPY para:

Consultei a página "Conditional Test Logic" do CATSPY: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic. A regra de detecção foi relacionada ao if existente em fake_import, que decidia se o import deveria prosseguir ou lançar ImportError. Usei a orientação de retirar a decisão condicional do teste como referência e adaptei a solução para um side_effect direto, adequado ao cenário específico em que a importação deveria falhar.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

O método utilizado foi remover a função fake_import, que possuía o if, e criar diretamente a exceção missing_docx = ImportError("simulated: python-docx not installed"). Em seguida, configurei essa exceção como side_effect do mock de builtins.__import__. Dessa forma, a tentativa de importação continuou produzindo ImportError, mas o teste deixou de possuir a função condicional. A asserção assert result is None foi preservada.

A08 – Conditional Test Logic

Eu estou atualmente trabalhando na refatoração do code smell Conditional Test Logic no teste tests/test_book_to_skill.py::TestDocxExtraction::test_extract_docx_validates_once_when_python_docx_unavailable, linha 1679, correspondente ao commit 8fe8cbd. A ocorrência estava em uma função que interceptava imports e utilizava um if para lançar uma exceção somente quando o módulo solicitado era docx.

Minhas principais dificuldades na remoção do code smell são:

Minha principal dificuldade ocorreu na primeira tentativa. A interceptação afetava todos os imports, incluindo imports internos e o fallback relacionado a cp437. Essa versão não foi commitada porque modificava mais elementos do ambiente do que o necessário.

Usei o CATSPY para:

Consultei a página "Conditional Test Logic" do CATSPY: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic. Comparei a definição do catálogo com o interceptador de imports que possuía um ramo exclusivo para o módulo docx. A seção sobre possíveis efeitos ajudou a avaliar o risco de um mock condicional alterar outros caminhos de execução. A orientação foi adaptada para mock.patch.dict(sys.modules, {"docx": None}), limitando a simulação apenas à dependência relevante.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

O método final foi usar mock.patch.dict(sys.modules, {"docx": None}). Dessa forma, somente o módulo docx foi marcado como indisponível. O fallback por zipfile, o texto extraído e a validação de segurança permaneceram preservados.

A09 – Conditional Test Logic

Eu estou atualmente trabalhando na refatoração do code smell Conditional Test Logic no

teste tests/test_book_to_skill.py::TestDetectStructure::test_persian_word_numerals_1_to_34, linha 921, correspondente ao commit c818e42. O teste utilizava um laço com uma asserção para cada numeral persa.

Minhas principais dificuldades na remoção do code smell são:

Minha principal dificuldade foi manter a correspondência exata entre os 34 ordinais e os números esperados. A mensagem individual associada a cada palavra deixou de existir, então foi necessário garantir que a lista completa permitisse detectar qualquer divergência.

Usei o CATSPY para:

Consultei a página "Conditional Test Logic" do CATSPY: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic. Usei a definição e a regra de detecção para relacionar o smell ao laço que executava uma asserção para cada ordinal persa. A página recomenda reduzir o fluxo variável dentro do teste. No projeto, essa orientação foi adaptada para calcular todos os resultados com map e realizar uma comparação única com os valores de 1 a 34.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

O método utilizado foi aplicar _chapter_number com map, formar a lista detected e compará-la com list(range(1, 35)). A verificação adicional da grafia com ZWNJ continuou separada. Essa mudança deve ser considerada uma reestruturação sintática, não uma melhoria comprovada de legibilidade.

A10 – Conditional Test Logic

Eu estou atualmente trabalhando na refatoração do code smell Conditional Test Logic no teste tests/test_book_to_skill.py::TestParserExceptionLogging::test_pypdf_warns_on_unexpecte

d_error_and_returns_none, linha 1798, correspondente ao commit fd8c848. O teste usava uma função de importação com um if para provocar uma falha relacionada ao pypdf.

Minhas principais dificuldades na remoção do code smell são:

Minha principal dificuldade foi executar a validação no ambiente local. O diretório temporário padrão estava inacessível. Repeti o teste utilizando --basetemp em uma pasta gravável e a execução passou.

Usei o CATSPY para:

Consultei a página "Conditional Test Logic" do CATSPY: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic. A definição foi relacionada ao if que tratava de maneira especial a importação de pypdf. A seção de possível efeito foi útil para avaliar que a interceptação global de imports poderia afetar operações não relacionadas ao cenário. A solução foi adaptada para um módulo falso inserido em sys.modules, no qual somente PdfReader lança a falha esperada.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

O método utilizado foi criar um módulo pypdf falso com MagicMock, configurar PdfReader.side_effect para lançar RuntimeError e inserir o módulo em sys.modules. O resultado continuou sendo None, e o aviso continuou sendo capturado.

A11 – Conditional Test Logic

Eu estou atualmente trabalhando na refatoração do code smell Conditional Test Logic no teste tests/test_corpus_newlines.py::test_corpus_writer_explicitly_disables_host_newline_translation, linha 86, correspondente ao commit d40ce32. A ocorrência estava em uma função auxiliar que utilizava um if para registrar somente a abertura do arquivo full_text.txt em modo de escrita.

Minhas principais dificuldades na remoção do code smell são:

Minha principal dificuldade foi substituir o registro manual sem perder a verificação do argumento newline="\n". Não bastava confirmar que Path.open tinha sido chamado, pois o teste precisava validar o caminho, o modo, a codificação e a configuração de nova linha.

Usei o CATSPY para:

Consultei a página "Conditional Test Logic" do CATSPY: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic. Comparei a regra de detecção com o if usado por record_open para decidir quais chamadas seriam registradas. A página destaca que condições podem dificultar a identificação do comportamento efetivamente validado. Usei essa orientação para substituir o registro condicional por um spy e verificar diretamente a chamada completa esperada.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

O método utilizado foi substituir a função por mock.patch.object com autospec, side_effect=real_open e um spy. Depois da execução, verifiquei a presença da chamada completa em open_spy.call_args_list.

A12 – Conditional Test Logic

Eu estou atualmente trabalhando na refatoração do code smell Conditional Test Logic no teste tests/test_hermes_host_support.py::test_hermes_extractor_probe_discovers_supported_layouts, linha 84, correspondente ao commit a11e705. O teste verificava se o caminho retornado era relativo e, nesse caso, adicionava o diretório nested.

Minhas principais dificuldades na remoção do code smell são:

Minha principal dificuldade foi confirmar o contrato real do script. Se o resultado correto

deveria ser sempre um caminho absoluto, corrigir um caminho relativo dentro do teste poderia ocultar um defeito.
Usei o CATSPY para: Consultei a página "Conditional Test Logic" do CATSPY: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic. A definição e o possível efeito foram relacionados ao if que corrigia o caminho quando ele não era absoluto. Essa condição poderia permitir que uma saída incorreta fosse ajustada pelo próprio teste. A orientação de deixar cada comportamento esperado explícito foi aplicada por meio de uma asserção direta de que o caminho retornado deve ser absoluto.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: O método utilizado foi remover o if e adicionar assert selected.is_absolute(). Em seguida, mantive a comparação entre o caminho resolvido e o extrator esperado. Com isso, o teste não corrige mais silenciosamente uma saída relativa.

A13 – Conditional Test Logic

Eu estou atualmente trabalhando na refatoração do code smell Conditional Test Logic no teste tests/test_hermes_host_support.py::test_hermes_project_candidates_require_enclosing_git_root, linha 287, correspondente ao commit 315e090. A preparação do cenário utilizava um laço para criar os extratores pessoal e local do projeto.
Minhas principais dificuldades na remoção do code smell são: Minha principal dificuldade foi garantir que ambos os arquivos continuassem sendo criados nos mesmos diretórios antes da execução do subprocesso. O teste depende da estrutura dos caminhos para verificar a exigência de um repositório Git envolvente.

Usei o CATSPY para: Consultei a página “Conditional Test Logic” do CATSPY: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic. A regra de detecção foi relacionada ao laço que criava os extratores pessoal e local. A seção de refatoração recomenda separar os diferentes casos quando o fluxo do teste depende de iterações ou condições. Adaptei essa recomendação para preparar cada extrator explicitamente, mantendo visível qual caminho representa a instalação pessoal e qual representa o projeto.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: O método utilizado foi desdobrar o laço. Criei explicitamente o diretório-pai e o arquivo do extrator pessoal e depois repeti as operações para o extrator local. Nenhum caminho ou arquivo do cenário foi alterado.

A14 – Conditional Test Logic

Eu estou atualmente trabalhando na refatoração do code smell Conditional Test Logic no teste tests/test_hermes_host_support.py::test_hermes_project_extractor_precedes_personal_installation, linha 111, correspondente ao commit 5a9dab3. O teste usava um laço para preparar dois extratores que participavam da regra de precedência.
Minhas principais dificuldades na remoção do code smell são: Minha principal dificuldade foi preservar a diferença semântica entre o extrator pessoal e o extrator instalado no projeto. Apesar de receberem as mesmas operações de criação, cada caminho representa uma opção diferente na seleção feita pelo script.
Usei o CATSPY para: Consultei a página “Conditional Test Logic” do CATSPY:

https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic. Usei a definição para classificar o laço que criava dois extratores participantes da regra de precedência. A recomendação de separar os casos foi aplicada ao desdobramento da preparação, permitindo visualizar individualmente o extrator pessoal e o extrator do projeto antes de verificar qual deles deve ser selecionado.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: O método utilizado foi substituir o laço pelas operações explícitas de criação dos dois arquivos. O extrator pessoal e o extrator local continuaram sendo preparados separadamente, e a asserção de que o projeto possui precedência foi mantida.

A15 – Conditional Test Logic

Eu estou atualmente trabalhando na refatoração do code smell Conditional Test Logic no teste tests/test_hermes_host_support.py::test_non_hermes_host_keeps_original_personal_precedence, linha 247, correspondente ao commit bce2634. A ocorrência era um laço que preparava o extrator pessoal e o extrator local antes de remover a variável HERMES_AGENT.
Minhas principais dificuldades na remoção do code smell são: Minha principal dificuldade foi manter o cenário específico de um host não Hermes. A retirada da variável de ambiente e a ordem de precedência não poderiam ser afetadas pela refatoração da preparação dos arquivos.
Usei o CATSPY para: Consultei a página “Conditional Test Logic” do CATSPY: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic. Relacionei a regra de detecção ao laço usado para criar as duas instalações antes de simular um host sem

HERMES_AGENT. A seção de refatoração serviu como referência para explicitar a preparação de cada caminho, sem modificar a configuração ambiental nem a expectativa de precedência da instalação pessoal.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: O método utilizado foi desdobrar a criação dos dois extratores em instruções explícitas. A remoção de HERMES_AGENT, a execução do script e a expectativa de escolha do extrator pessoal permaneceram inalteradas.

A16 – Conditional Test Logic

Eu estou atualmente trabalhando na refatoração do code smell Conditional Test Logic no teste tests/test_hermes_host_support.py::test_untrusted_hermes_project_cannot_override_personal_installation, linha 158, correspondente ao commit 4dd02c9. O teste utilizava um laço para criar o extrator pessoal e o extrator localizado em um projeto não confiável.
Minhas principais dificuldades na remoção do code smell são: Minha principal dificuldade foi manter visível a diferença de confiança entre os caminhos. O arquivo do projeto precisava existir para demonstrar que ele não poderia substituir a instalação pessoal, mesmo estando disponível.
Usei o CATSPY para: Consultei a página “Conditional Test Logic” do CATSPY: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic. A definição foi relacionada ao laço que criava o extrator pessoal e o extrator pertencente ao projeto não confiável. A orientação de separar os casos foi adaptada para criar cada arquivo explicitamente. Isso permitiu manter visível que os dois arquivos existem, mas somente o extrator pessoal pode ser selecionado.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

O método utilizado foi desdobrar o laço em operações explícitas para personal e untrusted. A lista de projetos confiáveis continuou vazia e a expectativa de seleção do extrator pessoal foi preservada.

A17 – Unknown Test

Eu estou atualmente trabalhando na refatoração do code smell Unknown Test no teste tests/test_book_to_skill.py::TestBatchResilience::test_extract_single_file_raises_on_missing, linha 288, correspondente ao commit 0acc256. O teste esperava ExtractionError e verificava a mensagem por meio do parâmetro match de pytest.raises.

Minhas principais dificuldades na remoção do code smell são:

Minha principal dificuldade foi tornar o resultado esperado mais explícito sem enfraquecer a validação. O teste precisava continuar falhando se nenhuma exceção fosse lançada ou se a mensagem não contivesse “File not found”.

Usei o CATSPY para:

Consultei a página “Unknown Test” do CATSPY: https://cat-spy.vercel.app/smells/Unknown%20Test. Utilizei as seções de definição, detecção e refatoração, que recomendam incluir asserções explícitas para demonstrar o resultado esperado. No teste analisado, pytest.raises já exigia ExtractionError, mas a mensagem estava incorporada ao parâmetro match. Adaptei a orientação para capturar a exceção em exc_info e verificar explicitamente a mensagem “File not found”.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

O método utilizado foi capturar a exceção em exc_info e adicionar assert exc_info.match("File not found"). O tipo e a mensagem esperados permaneceram os

mesmos.

A18 – Conditional Test Logic

Eu estou atualmente trabalhando na refatoração do code smell Conditional Test Logic no teste tests/test_sanitize_bidi_controls.py::TestSmugglingChannelsBeyondTheTagBlock::test_variation_selectors_are_stripped, linha 212, correspondente ao commit 3fbcb26. O teste percorria cinco seletores de variação e executava uma asserção para cada um.
Minhas principais dificuldades na remoção do code smell são: Minha principal dificuldade foi manter os cinco codepoints e suas mensagens corretas. Um erro de digitação poderia retirar um seletor do conjunto ou associar uma mensagem incorreta à asserção.
Usei o CATSPY para: Consultei a página "Conditional Test Logic" do CATSPY: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic. A regra de detecção foi relacionada ao laço que verificava cinco seletores de variação. A página recomenda evitar que diferentes verificações dependam de uma estrutura de controle dentro do teste. Adaptei essa orientação para cinco asserções explícitas, mantendo cada codepoint e sua mensagem de identificação.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: O método utilizado foi desdobrar o laço em cinco asserções explícitas, uma para cada codepoint. A parte posterior do teste, que cria um payload com seletores e valida a sanitização, permaneceu inalterada.

A19 – Conditional Test Logic

Eu estou atualmente trabalhando na refatoração do code smell Conditional Test Logic no teste tests/test_sanitize_bidi_controls.py::TestSmugglingChannelsBeyondTheTagBlock::test_scanner_flags_the_new_channels_too, linha 265, correspondente ao commit 129a164. A ocorrência era um laço que verificava cinco canais invisíveis no scanner.

Minhas principais dificuldades na remoção do code smell são:

Minha principal dificuldade foi preservar tanto os cinco valores Unicode quanto as mensagens explicativas. Cada asserção precisava continuar indicando qual codepoint não havia sido reconhecido.

Usei o CATSPY para:

Consultei a página "Conditional Test Logic" do CATSPY: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic. Usei a definição para identificar o laço que aplicava _is_invisible a cinco canais diferentes. A seção de possível efeito destaca a dificuldade de compreender quais caminhos foram realmente exercitados. A refatoração tornou cada canal visível por meio de uma asserção própria e de uma mensagem específica para o codepoint correspondente.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

O método utilizado foi substituir a iteração por cinco asserções explícitas de _is_invisible. Foram mantidos os codepoints FE0F, E0100, FFF9, 1D173 e 206A, além da mesma relação esperada entre scanner e extração.

A20 – Conditional Test Logic

Eu estou atualmente trabalhando na refatoração do code smell Conditional Test Logic no teste tests/test_sanitize_bidi_controls.py::TestDefaultIgnorableCarriers::test_visible_neighbours

_and_semantic_controls_are_kept, linha 304, correspondente ao commit 72e785c. O teste percorria a coleção CARRIER_NEIGHBOURS_KEPT e verificava que cada elemento permanecia visível.

Minhas principais dificuldades na remoção do code smell são:

Minha principal dificuldade foi desdobrar o laço sem criar uma verificação frágil caso a coleção mudasse. Para reduzir esse risco, também passei a validar que a coleção contém exatamente dez elementos.

Usei o CATSPY para:

Consultei a página "Conditional Test Logic" do CATSPY: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic. Relacionei a regra de detecção ao laço que percorria CARRIER_NEIGHBOURS_KEPT. A orientação de explicitar os casos foi aplicada por meio de uma asserção para cada posição. Como as verificações passaram a depender dos índices, também foi acrescentada uma asserção de tamanho para garantir que a coleção continuasse contendo exatamente dez elementos.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

O método utilizado foi adicionar a asserção de tamanho e criar uma asserção para cada posição, de zero a nove, com o codepoint correspondente. Os dez vizinhos continuaram sendo verificados como não invisíveis.

Aluna: Carla Cristina Sousa Araújo

B01 – Conditional Test Logic

Eu estou atualmente trabalhando na refatoração do code smell Conditional Test Logic no teste
tests/test_no_bytecode_pollution.py::test_entry_point_list_still_matches_the_source_tree, linha 210, correspondente ao commit d618dfb. O teste possuía um if que decidia se um

arquivo deveria ser adicionado ao conjunto de entry points encontrados.

Minhas principais dificuldades na remoção do code smell são:

Minha principal dificuldade foi retirar a decisão do corpo do teste sem alterar o critério de identificação do MAIN_GUARD. Também percebi que mover a condição não significa necessariamente eliminar a lógica condicional do arquivo.

Usei o CATSPY para:

Consultei a página "Conditional Test Logic" do CATSPY: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic. A definição foi relacionada ao if que decidia se um arquivo deveria ser adicionado ao conjunto de entry points. A seção de refatoração recomenda extrair blocos condicionais para métodos separados. Apliquei essa recomendação por meio de _guarded_entry_point, embora tenha observado que a condição foi isolada em um helper e não completamente eliminada do arquivo.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

O método utilizado foi criar o helper _guarded_entry_point, que retorna um conjunto contendo o caminho quando encontra o guard ou um conjunto vazio nos demais casos. O teste passou a atualizar found com o resultado do helper.

B02 – Conditional Test Logic

Eu estou atualmente trabalhando na refatoração do code smell Conditional Test Logic no teste tests/test_no_bytecode_pollution.py::test_entry_points_set_the_policy_before_importing_the_package, linhas 182 e 183, correspondente ao commit 9fae83b. A ocorrência estava em uma expressão geradora condicional usada para localizar a primeira importação de book_to_skill.

Minhas principais dificuldades na remoção do code smell são:

Minha principal dificuldade foi preservar a regra de busca e o comportamento caso nenhuma importação fosse encontrada. O índice retornado ainda precisava representar a primeira ocorrência válida.

Usei o CATSPY para:

Consultei a página "Conditional Test Logic" do CATSPY: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic. Usei a regra de detecção para relacionar o smell à expressão geradora condicional que localizava a primeira importação de book_to_skill. A recomendação de extrair a lógica foi adaptada para _package_import_index, centralizando a busca e deixando no teste apenas a utilização do índice encontrado.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

O método utilizado foi criar _package_import_index. O helper percorre as linhas, retorna o índice da primeira importação do pacote e lança ValueError quando não encontra nenhuma. O teste passou a chamar o helper diretamente. A lógica foi isolada, mas não deixou de existir, o que considerei uma limitação da abordagem.

B03 – Conditional Test Logic

Eu estou atualmente trabalhando na refatoração do code smell Conditional Test Logic no teste tests/test_sanitize_bidi_controls.py::TestScannerAndExtractorAgree::test_visible_characters_are_not_flagged, linha 201, correspondente ao commit f1f7584. O teste percorria uma string com caracteres visíveis e executava uma asserção para cada caractere.

Minhas principais dificuldades na remoção do code smell são:

Minha principal dificuldade foi preservar caracteres de alfabetos diferentes, além de espaço, tabulação e quebra de linha. Todos precisavam continuar sendo avaliados.

Usei o CATSPY para:

Consultei a página "Conditional Test Logic" do CATSPY: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic. A regra de detecção foi relacionada ao laço que verificava caracteres visíveis de diferentes alfabetos. O catálogo aponta que estruturas de controle podem dificultar a identificação dos resultados esperados. Adaptei a orientação para reunir em flagged qualquer caractere incorretamente classificado e verificar explicitamente que essa coleção permanece vazia.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

O método utilizado foi aplicar filter para reunir os caracteres incorretamente classificados como invisíveis e comparar o resultado com uma lista vazia. O mesmo conjunto de caracteres continuou sendo testado. Considero essa alteração principalmente sintática.

B04 – Conditional Test Logic

Eu estou atualmente trabalhando na refatoração do code smell Conditional Test Logic no teste tests/test_no_bytecode_pollution.py::test_entry_points_set_the_policy_before_importing_the_package, linha 173, correspondente ao commit 0d1f7bd. O teste percorria todos os entry points e executava diferentes verificações dentro do laço.

Minhas principais dificuldades na remoção do code smell são:

Minha principal dificuldade apareceu na CI. A primeira versão do helper usava a anotação str | None, aceita no meu Python 3.13, mas incompatível com Python 3.9. Removi a anotação incompatível e executei novamente o teste na matriz da CI. A versão corrigida passou em Python 3.9, 3.10, 3.11, 3.12 e 3.13.

Usei o CATSPY para: Consultei a página “Conditional Test Logic” do CATSPY: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic. Comparei a definição do catálogo com o laço que verificava o guard, a posição do import e a política de bytecode de cada entry point. A recomendação de extrair blocos condicionais foi adaptada para _entry_point_policy_error, permitindo que o teste reúna todas as mensagens e verifique que nenhuma violação foi encontrada.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: O método utilizado foi criar _entry_point_policy_error, que retorna uma mensagem quando encontra um problema e None quando a política está correta. O teste passou a aplicar o helper a todos os entry points e verificar que a lista de erros estava vazia.

B05 – Conditional Test Logic

Eu estou atualmente trabalhando na refatoração do code smell Conditional Test Logic no teste tests/test_openclaw_host_support.py::test_openclaw_extractor_probe_discovers_supported_layouts, linha 97, correspondente ao commit 2c2eb1f. O teste utilizava um if para acrescentar o diretório nested quando o caminho retornado não era absoluto.
Minhas principais dificuldades na remoção do code smell são: Minha principal dificuldade foi confirmar o comportamento de pathlib ao combinar o diretório-base com caminhos relativos e absolutos. A nova expressão deveria funcionar nos dois casos.
Usei o CATSPY para: Consultei a página “Conditional Test Logic” do CATSPY:

https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic. Relacionei a definição ao ramo que distinguia caminhos absolutos de relativos. A seção de possível efeito ajudou a avaliar que os dois caminhos de execução deveriam produzir a mesma localização final. A condição foi substituída por uma composição direta com pathlib, cuja resolução preserva caminhos absolutos e completa caminhos relativos.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: O método utilizado foi construir (nested / _host_path(result.stdout)).resolve() e comparar o resultado com o caminho resolvido do extrator. Quando o segundo caminho é absoluto, pathlib preserva o caminho absoluto; quando é relativo, usa nested como base.

B06 – Conditional Test Logic

Eu estou atualmente trabalhando na refatoração do code smell Conditional Test Logic no teste tests/test_pdf_page_number_detection.py::test_every_canonical_numeral_1_to_99_matches, linha 56, correspondente ao commit fb4d3eb. O teste percorria os valores de 1 a 99, construía o numeral romano e verificava cada correspondência.
Minhas principais dificuldades na remoção do code smell são: Minha principal dificuldade foi manter a formação correta dos numerais a partir das listas de dezenas e unidades. Também precisei preservar a cobertura completa dos 99 valores.
Usei o CATSPY para: Consultei a página “Conditional Test Logic” do CATSPY: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic. A regra de detecção foi relacionada ao laço que construía e verificava todos os numerais romanos entre 1 e 99. A orientação de reduzir o fluxo iterativo dentro do teste foi adaptada para gerar os pares com map, reunir as correspondências ausentes e verificar que a lista resultante está vazia.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

O método utilizado foi criar os pares de valor e numeral com map, filtrar os casos que não correspondessem ao padrão e verificar que a lista misses estava vazia. A cobertura permaneceu igual, mas considero a mudança uma reexpressão voltada ao detector, sem melhoria de legibilidade comprovada.

B07 – Conditional Test Logic

Eu estou atualmente trabalhando na refatoração do code smell Conditional Test Logic no teste tests/test_pdf_page_number_detection.py::test_letters_outside_the_1_to_99_range_are_kept, linha 63, correspondente ao commit 06d20e4. O teste percorria seis letras e verificava que nenhuma era reconhecida como número de página.

Minhas principais dificuldades na remoção do code smell são:

Minha principal dificuldade foi preservar as versões maiúsculas e minúsculas de C, D e M. Também avaliei criticamente se a nova construção realmente ficaria mais clara.

Usei o CATSPY para:

Consultei a página “Conditional Test Logic” do CATSPY: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic. A definição foi relacionada ao laço que verificava as formas maiúsculas e minúsculas de C, D e M. A página foi utilizada para confirmar a classificação e o possível efeito sobre a compreensão do teste. A solução com filter foi uma adaptação local e não uma reprodução do exemplo apresentado pelo catálogo.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

O método utilizado foi filtrar as letras que correspondessem indevidamente ao padrão e

verificar que matched estava vazia. O TEMPY deixou de apontar a ocorrência, mas a solução com filter e lambda não representa necessariamente uma melhoria de legibilidade.

B08 – Conditional Test Logic

Eu estou atualmente trabalhando na refatoração do code smell Conditional Test Logic no teste tests/test_pdf_page_number_detection.py::test_non_numeral_words_still_rejected, linha 74, correspondente ao commit 311cb8b. O teste percorria as entradas Chapter, Introduction, the e 42a, verificando individualmente que não eram numerais válidos.

Minhas principais dificuldades na remoção do code smell são:

Minha principal dificuldade foi conservar exatamente essas quatro entradas, pois elas representam tipos diferentes de falso positivo.

Usei o CATSPY para:

Consultei a página "Conditional Test Logic" do CATSPY: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic. Usei a regra de detecção para identificar o laço que verificava quatro tipos de entradas não numéricas. A seção sobre possível efeito foi considerada ao preservar separadamente palavras comuns, um título e uma combinação alfanumérica. A refatoração passou a reunir apenas as correspondências indevidas e a exigir que a lista final estivesse vazia.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

O método utilizado foi reunir as palavras, filtrar as que correspondessem indevidamente à expressão regular e comparar a lista com []. O comportamento foi preservado, embora a alteração seja principalmente sintática.

B09 – Conditional Test Logic

Eu estou atualmente trabalhando na refatoração do code smell Conditional Test Logic no teste tests/test_repo_hygiene.py::test_no_compiled_bytecode_is_tracked, linhas 55 e 56, correspondente ao commit cbda79b. O teste utilizava uma compreensão condicional para selecionar arquivos .pyc ou caminhos contendo __pycache__/.

Minhas principais dificuldades na remoção do code smell são:

Minha principal dificuldade foi preservar a condição composta com or. Uma alteração incorreta poderia deixar de detectar uma das duas formas de bytecode versionado.

Usei o CATSPY para:

Consultei a página "Conditional Test Logic" do CATSPY: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic. Relacionei a regra de detecção à compreensão que possuía dois critérios condicionais: extensão .pyc e presença de __pycache__/. A página serviu para confirmar a classificação. A compreensão foi substituída por filter, preservando os mesmos critérios e a mesma mensagem de falha.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

O método utilizado foi substituir a compreensão por filter com uma função lambda que mantém exatamente os dois critérios. A lista offenders continuou sendo comparada com uma lista vazia e a mensagem de falha permaneceu inalterada.

B10 – Unknown Test

Eu estou atualmente trabalhando na refatoração do code smell Unknown Test no teste tests/test_book_to_skill.py::TestBatchResilience::test_extract_single_file_raises_on_unsupported, linha 291, correspondente ao commit ccce50c.

O teste verifica o comportamento de extract_single_file quando recebe um arquivo com uma extensão não suportada. Antes da refatoração, o tipo da exceção e o conteúdo da

mensagem eram verificados juntos pelo parâmetro match de pytest.raises. Embora essa construção fosse funcional, o TEMPY classificou o teste como Unknown Test porque a asserção sobre o conteúdo da exceção não aparecia separadamente no corpo do teste.

Minhas principais dificuldades na remoção do code smell são:

Minha principal dificuldade foi tornar a verificação mais explícita sem enfraquecer o teste. Não seria suficiente verificar somente que algum ExtractionError foi lançado, pois o teste também precisava confirmar que a falha ocorreu especificamente porque o formato do arquivo não era suportado.
Também precisei preservar o comportamento de pytest.raises: o teste deveria continuar falhando se nenhuma exceção fosse lançada, se fosse lançada uma exceção de outro tipo ou se a mensagem não correspondesse a “Unsupported format”.

Usei o CATSPY para:

Consultei a página “Unknown Test” do CATSPY: https://cat-spy.vercel.app/smells/Unknown%20Test. A definição informa que a ausência de uma asserção clara pode fazer um teste passar apenas porque nenhuma falha inesperada ocorreu. A seção de refatoração recomenda introduzir verificações explícitas do resultado esperado. Adaptei essa orientação para capturar ExtractionError em exc_info e verificar separadamente a mensagem “Unsupported format”.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

Substituí a utilização de pytest.raises(ExtractionError, match="Unsupported format") pela captura da exceção com pytest.raises(ExtractionError) as exc_info.
Depois da execução da função, adicionei a asserção assert exc_info.match("Unsupported format"). Dessa forma, o tipo da exceção continua sendo validado pelo gerenciador de contexto e sua mensagem passa a ser verificada explicitamente.
A refatoração preservou o arquivo de entrada, a chamada de extract_single_file, o tipo

ExtractionError e a mensagem esperada. O teste continuou passando após a alteração.

B11 – Conditional Test Logic

Eu estou atualmente trabalhando na refatoração do code smell Conditional Test Logic no teste tests/test_rtf_destination_groups.py::TestDestinationGroupsDropped::test_chapter_heading_is_on_its_own_line, linha 59, correspondente ao commit 44ea1a4. O teste utilizava uma compreensão com condição para eliminar linhas vazias do resultado RTF.

Minhas principais dificuldades na remoção do code smell são:

Minha principal dificuldade foi preservar o comportamento de ln.strip(): a linha deveria ser mantida quando possuísse qualquer conteúdo diferente de espaços, mesmo que o texto original conservasse espaços nas extremidades.

Usei o CATSPY para:

Consultei a página "Conditional Test Logic" do CATSPY: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic. A regra de detecção foi relacionada à compreensão que incluía uma linha somente quando ln.strip() retornava conteúdo. A página ajudou a confirmar que a filtragem condicional estava incorporada à preparação do resultado. A refatoração utilizou filter(str.strip, ...), preservando a exclusão das linhas vazias e a asserção sobre o primeiro título.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

O método utilizado foi substituir a compreensão por filter(str.strip, ...). A filtragem continuou considerando o conteúdo sem espaços para decidir se a linha deveria permanecer, e a asserção sobre "Chapter 1" continuou igual.

B12 – Conditional Test Logic

Eu estou atualmente trabalhando na refatoração do code smell Conditional Test Logic no teste tests/test_sanitize_bidi_controls.py::TestSmugglingChannelsBeyondTheTagBlock::test_annotation_and_format_controls_are_stripped, linha 224, correspondente ao commit 3f63875. O teste percorria oito controles Unicode e verificava que todos eram invisíveis.

Minhas principais dificuldades na remoção do code smell são:

Minha principal dificuldade foi manter os três grupos de codepoints: anotações interlineares, controles musicais e controles de formatação obsoletos.

Usei o CATSPY para:

Consultei a página "Conditional Test Logic" do CATSPY: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic. A definição foi relacionada ao laço que verificava controles de anotação, controles musicais e controles de formatação obsoletos. A orientação de explicitar o resultado esperado foi adaptada para reunir os codepoints que fossem incorretamente considerados visíveis e verificar que essa lista permanece vazia.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

O método utilizado foi guardar os codepoints em uma tupla, filtrar aqueles que fossem incorretamente considerados visíveis e verificar que a lista estava vazia. Nenhum valor Unicode foi adicionado ou removido.

B13 – Conditional Test Logic

Eu estou atualmente trabalhando na refatoração do code smell Conditional Test Logic no teste tests/test_sanitize_bidi_controls.py::TestDefaultIgnorableCarriers::test_carriers_are_stripped, linha 280, correspondente ao commit 2c4c047. Para cada carrier, o teste verificava sua

classificação e o resultado da sanitização de uma string.

Minhas principais dificuldades na remoção do code smell são:

Minha principal dificuldade foi reunir duas expectativas diferentes sem perder nenhuma delas. Cada codepoint deveria ser invisível e sua remoção deveria transformar a entrada em ("ab", 1).

Usei o CATSPY para:

Consultei a página “Conditional Test Logic” do CATSPY: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic. Comparei a regra de detecção com o laço que executava duas verificações para cada carrier. A seção de possível efeito ajudou a manter explícitas as duas propriedades esperadas: o codepoint deve ser invisível e sua remoção deve produzir ("ab", 1). A refatoração reúne somente os elementos que violem uma dessas propriedades.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

O método utilizado foi filtrar os codepoints que falhassem em pelo menos uma das duas condições e verificar que failures estava vazia. Os dois critérios originais continuaram presentes na expressão.

B14 – Conditional Test Logic

Eu estou atualmente trabalhando na refatoração do code smell Conditional Test Logic no teste tests/test_sanitize_bidi_controls.py::TestSmugglingChannelsBeyondTheTagBlock::test_neighbours_of_the_selector_ranges_are_kept, linha 220, correspondente ao commit 14cd1f9. O teste percorria quatro codepoints vizinhos às faixas de seletores e verificava que continuavam visíveis.

Minhas principais dificuldades na remoção do code smell são: Minha principal dificuldade foi manter a distinção entre os limites testados e os próprios seletores invisíveis. Um valor incorreto poderia mudar o sentido do teste.
Usei o CATSPY para: Consultei a página "Conditional Test Logic" do CATSPY: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic. A regra de detecção foi relacionada à repetição que verificava os quatro limites vizinhos às faixas de seletores. A seção de possível efeito foi considerada porque confundir um limite visível com um seletor invisível alteraria a finalidade do teste. A solução reúne apenas classificações incorretas e exige que a lista esteja vazia.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: O método utilizado foi reunir os quatro codepoints, filtrar aqueles classificados incorretamente como invisíveis e verificar que invisible estava vazia. Os mesmos limites continuaram sendo utilizados.

B15 – Conditional Test Logic

Eu estou atualmente trabalhando na refatoração do code smell Conditional Test Logic no teste tests/test_sanitize_bidi_controls.py::TestScannerAndExtractorAgree::test_previously_covered_codepoints_still_covered, linha 196, correspondente ao commit 344f1d5. O teste mantinha uma rede de regressão para codepoints cobertos por correções anteriores.
Minhas principais dificuldades na remoção do code smell são: Minha principal dificuldade foi não remover nenhum dos oito valores históricos, pois isso diminuiria a proteção contra regressões.

Usei o CATSPY para: Consultei a página “Conditional Test Logic” do CATSPY: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic. A definição foi relacionada ao laço que mantinha a cobertura de oito codepoints históricos. A página foi utilizada para confirmar a classificação e avaliar a necessidade de manter todos os casos exercitados. A refatoração preservou integralmente o conjunto e passou a reunir somente os valores incorretamente considerados visíveis.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: O método utilizado foi criar uma tupla com os mesmos codepoints, filtrar os que fossem considerados visíveis e verificar que o resultado estava vazio. A finalidade de regressão e o conjunto testado permaneceram iguais.

B16 – Conditional Test Logic

Eu estou atualmente trabalhando na refatoração do code smell Conditional Test Logic no teste tests/test_sanitize_bidi_controls.py::TestScannerAndExtractorAgree::test_scanner_flags_everything_extraction_strips, linha 181, correspondente ao commit dab727c. O teste percorria os controles bidirecionais e outros invisíveis e comparava o sanitizador com o scanner.
Minhas principais dificuldades na remoção do code smell são: Minha principal dificuldade foi preservar as duas verificações para todos os codepoints. O teste deveria detectar tanto uma falha do sanitizador quanto uma divergência do scanner.
Usei o CATSPY para: Consultei a página “Conditional Test Logic” do CATSPY:

https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic. Relacionei a definição ao laço que verificava dois predicados para cada codepoint: o sanitizador e o scanner. A seção de possível efeito ajudou a preservar a intenção de detectar divergências entre os componentes. A solução reúne os valores que falhem em qualquer um dos predicados e verifica que não existem falhas.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: O método utilizado foi converter os caracteres em codepoints com map, filtrar os valores que falhassem em qualquer um dos dois predicados e verificar que failures estava vazia. A paridade entre os componentes continuou sendo o objetivo do teste.

B17 – Conditional Test Logic

Eu estou atualmente trabalhando na refatoração do code smell Conditional Test Logic no teste tests/test_sanitize_bidi_controls.py::TestDefaultIgnorableCarriers::test_scanner_flags_the_carriers, linha 294, correspondente ao commit c324b6b. O teste percorria todos os carriers e verificava que o scanner os classificava como invisíveis.
Minhas principais dificuldades na remoção do code smell são: Minha principal dificuldade foi preservar o conjunto completo DEFAULT_IGNORABLE_CARRIERS, sem substituir a coleção por uma amostra menor.
Usei o CATSPY para: Consultei a página “Conditional Test Logic” do CATSPY: https://cat-spy.vercel.app/smells/Conditional%20Test%20Logic. A regra de detecção foi relacionada ao laço que aplicava _is_invisible a todos os elementos de DEFAULT_IGNORABLE_CARRIERS. A orientação de explicitar o resultado foi adaptada para formar a lista unflagged, que deve permanecer vazia. A coleção completa

continuou sendo utilizada.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: O método utilizado foi filtrar os carriers que não fossem sinalizados pelo scanner e verificar que unflagged estava vazia. Todos os elementos da coleção original continuaram participando da validação.

B18 – Programming Paradigms Blend

Eu estou atualmente trabalhando na refatoração do code smell Programming Paradigms Blend no arquivo tests/test_cjk_supplementary_plane.py, nas linhas 19 e 29, correspondente ao commit 0bcf660. O arquivo misturava alteração procedural de sys.path, constantes globais e classes de teste.
Minhas principais dificuldades na remoção do code smell são: Minha principal dificuldade foi reorganizar as amostras CJK sem alterar as strings Unicode ou o número de caracteres usado nos testes. Também precisei confirmar que o projeto já disponibilizava o pacote sem a alteração manual de sys.path.
Usei o CATSPY para: Consultei a página “Programming Paradigms Blend” do CATSPY: https://cat-spy.vercel.app/smells/Programming%20Paradigms%20Blend. A página define o smell como a mistura de elementos globais ou procedurais com classes de teste e recomenda encapsular os dados dentro da classe, em setUp ou em estruturas auxiliares. Relacionei essa definição às constantes globais BMP e SIP e à manipulação de sys.path. A orientação foi adaptada para uma classe compartilhada _CjkSamples.
Eu estou usando os seguintes métodos de refatoração para remover o code smell:

O método utilizado foi remover a manipulação de sys.path, criar a classe compartilhada _CjkSamples e fazer as duas classes de teste herdarem dela. As referências passaram de BMP e SIP globais para self.BMP e self.SIP.

B19 – Exception Handling

Eu estou atualmente trabalhando na refatoração do code smell Exception Handling no teste tests/test_book_to_skill.py::TestEpubExtractionFix::test_epub_no_tuple_unpack_error, linha 211, correspondente ao commit 06ee55a. O teste envolvia a extração em um try/except que capturava TypeError e ValueError para chamar pytest.fail.

Minhas principais dificuldades na remoção do code smell são:

Minha principal dificuldade foi confirmar que a remoção não enfraqueceria o teste. Verifiquei que qualquer uma dessas exceções, quando inesperada e não capturada, já faz o pytest registrar a execução como falha.

Usei o CATSPY para:

Consultei a página "Exception Handling" do CATSPY: https://cat-spy.vercel.app/smells/Exception%20Handling. A página descreve o uso manual de blocos try-except para aprovar ou reprovar testes e recomenda utilizar o comportamento nativo do framework. No caso analisado, nenhuma exceção era esperada. Por isso, em vez de usar pytest.raises, removi o tratamento manual e deixei que o próprio pytest registrasse qualquer TypeError ou ValueError inesperado como falha.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

O método utilizado foi remover o try/except e executar extract_single_file diretamente dentro do patch. A asserção posterior sobre o texto extraído foi mantida. Exceções inesperadas continuam falhando naturalmente.



UNIVERSIDADE
FEDERAL DO CEARÁ

B20 – Unknown Test

Eu estou atualmente trabalhando na refatoração do code smell Unknown Test no teste tests/evals/test_manifest.py::test_invalid_configs_fail_clearly, linha 80, correspondente ao commit 0d26994. O teste recebia diferentes configurações inválidas e validava a mensagem pelo parâmetro match de pytest.raises.
Minhas principais dificuldades na remoção do code smell são: Minha principal dificuldade foi preservar a parametrização. Cada execução recebe uma mutação e uma expressão esperada diferentes, portanto a nova asserção deveria continuar utilizando o valor expected correspondente.
Usei o CATSPY para: Consultei a página "Unknown Test" do CATSPY: https://cat-spy.vercel.app/smells/Unknown%20Test. Usei principalmente as seções de definição, detecção e refatoração, que recomendam apresentar verificações explícitas do resultado esperado. No teste parametrizado, a exceção já era exigida por pytest.raises, mas sua mensagem estava incorporada ao parâmetro match. A orientação foi adaptada para capturar ValueError e verificar explicitamente exc_info.match(expected).
Eu estou usando os seguintes métodos de refatoração para remover o code smell: O método utilizado foi capturar ValueError em exc_info e executar assert exc_info.match(expected). O teste continuou parametrizado, exigindo a mesma exceção e a mesma mensagem para cada configuração inválida.